

Generative quantum advantage for classical and quantum problems

Quantum Machine Learning

April 10, 2026

Beyond-Classical Sampling: Status and Gap

Established fact

Beyond-classical quantum experiments have shown that quantum devices can sample from distributions that are classically intractable to simulate.

Critical gap

Complexity barriers have thwarted efficient learning of such distributions.

A Stronger Definition of Generative Quantum Advantage

Goal emphasized in this talk

Generative quantum advantage: quantum computers can **learn and generate** desired outputs substantially better than classical computers, not just sample once from a hard distribution.

Advantage loop = $\underbrace{\text{Trainable}}_{\text{efficient learning}} + \underbrace{\text{Inferable}}_{\text{efficient generation of classically hard distributions}}$

Main Results

- Introduce families of generative quantum models that are:
 - classically hard to simulate,
 - efficiently trainable.
- Demonstrate on a 68-qubit superconducting quantum processor in two scenarios:
 - ① learning classically intractable probability distributions,(bitstring generation task)
 - ② learning quantum circuits for accelerated physical simulation.
- The two tasks leverage different learning algorithms.

Task 1 : Inference with DQNN/IDQNN

Inference

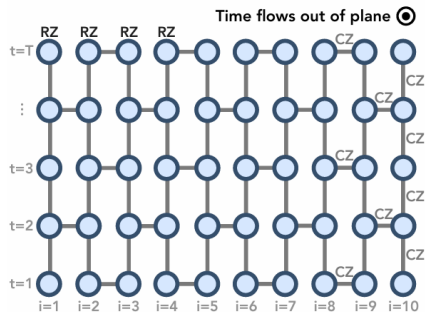
Given an input bitstring $x \in (0, 1)^n$, the generative quantum model probabilistically generates output bitstring $y \in (0, 1)^n$ according to n model parameters θ :

$$y \sim p_{\theta}(y | x).$$

- Models: DQNN(Deep Quantum Neural Network) and shallow IDQNN(Instantaneous Deep Quantum Neural Network).
- Possess the ability to generate classically intractable distributions.

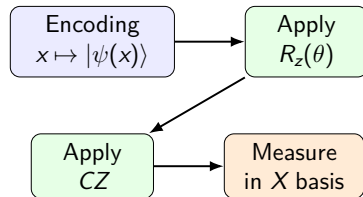
We had proved that $P_{IDQNN}(y|x, \theta) = P_{DQNN}(y|x, \theta)$.

IDQNN Definition and Architecture



2D Shallow QNN

2D shallow QNN architecture used by IDQNN.



$$|\psi_{\text{out}}\rangle = (\prod CZ) (\prod R_z(\theta)) |\psi(x)\rangle, \quad y \sim p_{\theta}(y | x)$$

Input-state encoding rule

Qubit i is initialized from $\{|0\rangle, |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\}$ according to bit x_i : $0 \mapsto |+\rangle$, $1 \mapsto |0\rangle$.

Task 1 Learning Algorithm: Dataset Construction

- 1 Sample N_{sample} input bitstrings from $D(x)$.
- 2 Fix ideal parameters θ_{real} .
- 3 Generate the dataset under ideal parameters:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N_{\text{sample}}}.$$

Training objective

Estimate θ from $\mathcal{D}$ such that θ approaches θ_{real} .

Task 1 Learning Algorithm: Block-wise Estimation

Core idea: estimate each parameter position block-wise.

- For each parameter at position (i, j) , filter samples satisfying:

$$x_{i,j} = 0, \quad x_{\mathcal{N}(i,j)} = 1,$$

where $\mathcal{N}(i, j)$ denotes positions connected via CZ gates.

- Compute the parameter estimate from the filtered samples using the prescribed formula.

$$\hat{\theta}_{i,j} = \arccos \left(1 - \frac{2}{N_{\text{sp}}} \sum_{t=1}^{N_{\text{sp}}} y_{(t)}^{(i,j)} \right).$$

Proof Sketch (IDQNN)

- Focus on parameter at position (i, j) .
- Filtering condition: $x_{i,j} = 0$ and neighbors in $\mathcal{N}(i, j)$ are 1.
- This simplifies the local input to

$$|0\rangle_{i,j-1} |+\rangle_{i,j} |0\rangle_{i,j+1}.$$

- Measure qubit (i, j) in the X basis.

$$|\psi'\rangle = |0\rangle_{i,j-1} \left(|+\rangle_{i,j} \cos \frac{\theta_{i,j}}{2} - |-\rangle_{i,j} \sin \frac{\theta_{i,j}}{2} \right) |0\rangle_{i,j+1}$$

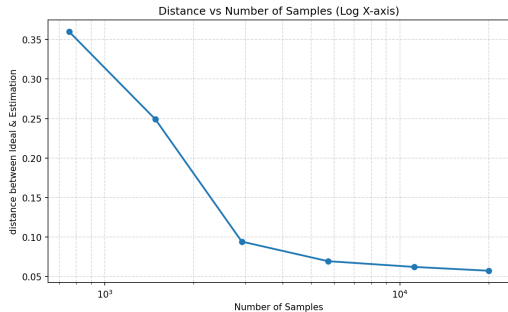
$$\Pr(y_{i,j} = 1) = \frac{1 - \cos \theta_{i,j}}{2}$$

Estimator form

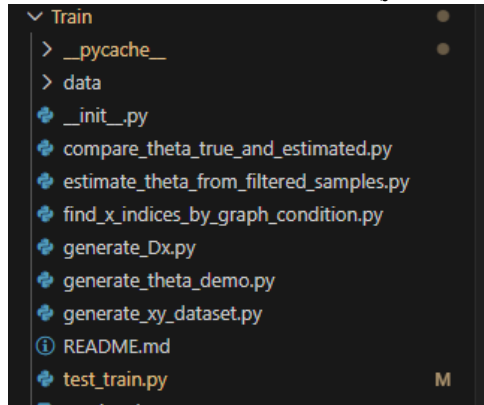
$$\hat{\theta}_{i,j} = \arccos \left(1 - \frac{2}{N_{\text{sp}}} \sum_{t=1}^{N_{\text{sp}}} y_{(t)}^{(i,j)} \right)$$

Code Result

I set $n = 12$; $n_1 = 3$, $m = 4$; the distance is defined as $d = \max |\cos \hat{\theta}_{i,j} - \cos \theta_{i,j}^{\text{real}}|$.



Code result: distance vs number of samples (log-scale x-axis).



Python implementation screenshot/result.

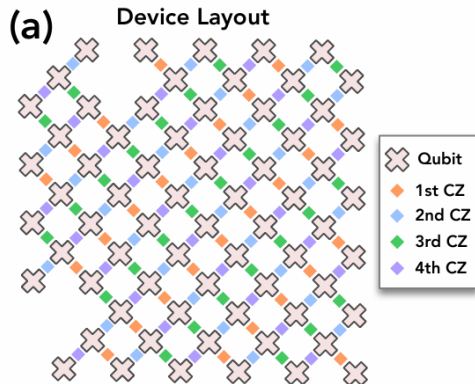
Hardware Experiment Setting (Authors' Result)

- The authors implement deep circuit forms on a superconducting quantum processor:
 - 68 physical qubits,
 - 12 rounds (layers) of execution.
- This corresponds to a shallow equivalent structure on 816 qubits (68×12).

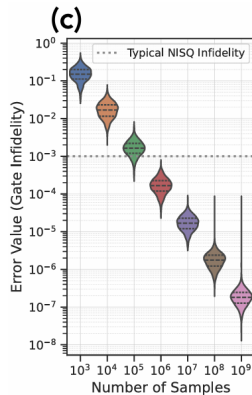
Implementation trick

They bypass mid-circuit measurement by evaluating hardware only for $x = 000 \cdots 0$ and randomly initializing all but the final layer of y .

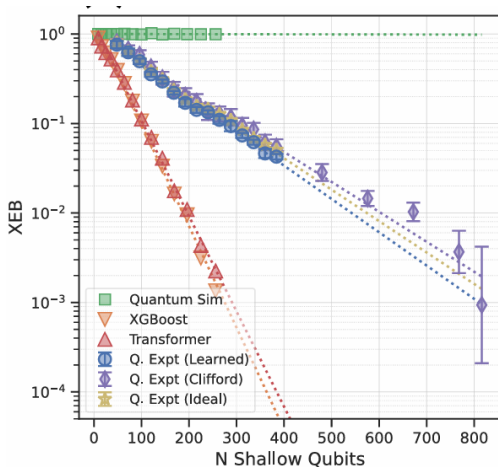
Device Mapping and Errors



(a) Around-CZ-gates device layout: 68 physical qubits, mapped to up to 816 shallow qubits.



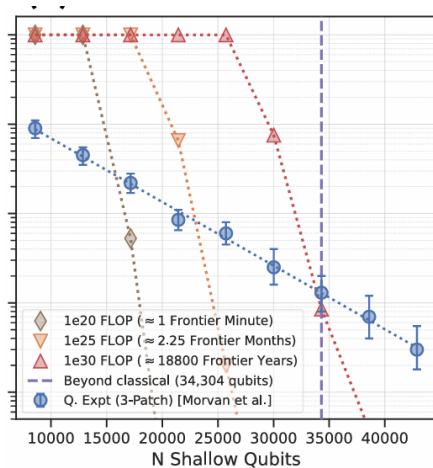
XEB Performance on Hardware



$$\text{XEB} = 2^n \frac{1}{N} \sum_{i=1}^N P_{\text{ideal}}(y_i) - 1$$

- Quantum and classical models are compared using Cross-Entropy Benchmarking score.
- Inference is evaluated at input bitstring $x = 000 \dots 0$.
- The same datasets are supplied to classical models (Transformer and XGBoost)

Beyond-Classical Regime



- Performance is inferred in the beyond-classical regime.